Deep models — a primer

Lecture 2 – From a single neuron to a network

Dr Thomas Robinson

August 4, 2026

LSE SUMMER SCHOOL · 2026

Part 1 · One neuron, slowly

The name is a metaphor

The word “neural network” comes from a 1940s analogy to biological neurons:

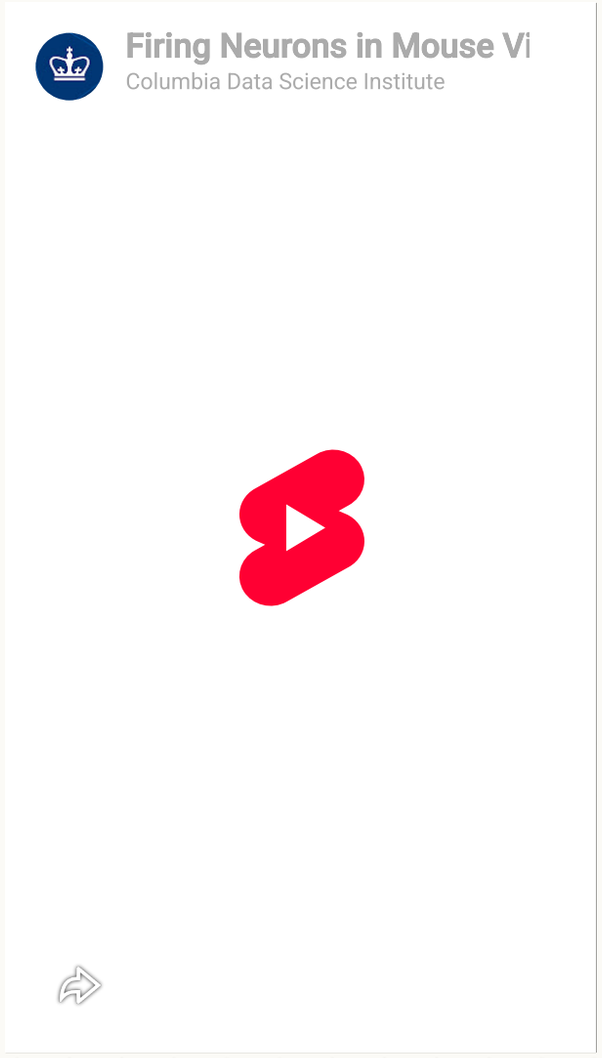
- A neuron receives signals from upstream cells.
- It “decides” whether to fire — an all-or-nothing action.
- Its threshold for firing shifts with experience.

A *very loose* abstraction:

- Take some input
- Weight it
- Apply a threshold
- Pass on the result.

> ⚠ **Warning**
>
> The metaphor is **historical**, not scientific. Modern deep learning is not trying to be neuroscience.
>
> The name is sticky. You can think of every “neuron” in this course as just *“a particular small computation with weights”*.

A real neuron



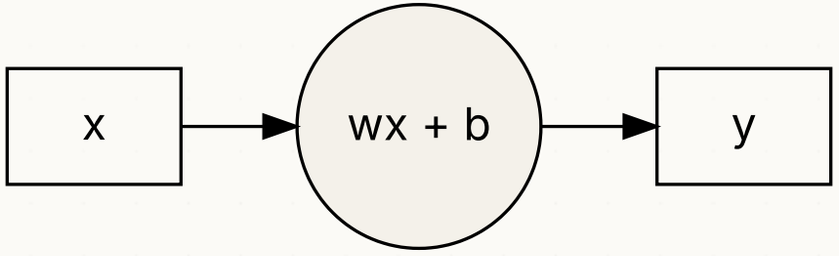
The linear perceptron

A single input x goes in. A single number y comes out:

$$y = f(x) = wx + b$$

TWO TOGGLES WE CAN ADJUST

- w — the **weight**. Scales the input.
- b — the **bias**. Shifts the weighted result.



That’s it. Two numbers!

Let’s run a number through it

Pick concrete values: $w = 2$, $b = 1$.

If $x = 3$:

$$wx = 2 \times 3 = 6$$
$$y = wx + b = 6 + 1 = 7$$

TRY IT YOURSELF

What is y if $w = 0.5, b = -1, x = 4$?

ANSWER

$y = (0.5 \times 4) + (-1) = 2 - 1 = 1.$

Forward propagation

We pushed an input value x (the data) through the model from left to right.

We call this **forward propagation** (or the **forward pass**).

Every neural network works the same way:

1. Feed input data in
2. Compute layer-by-layer
3. Get an output.

STOP & CHECK

So far, we have computed exactly one prediction using a single input value

- We can scale this up with some maths

Multiple inputs

What if we have several features, not just one?

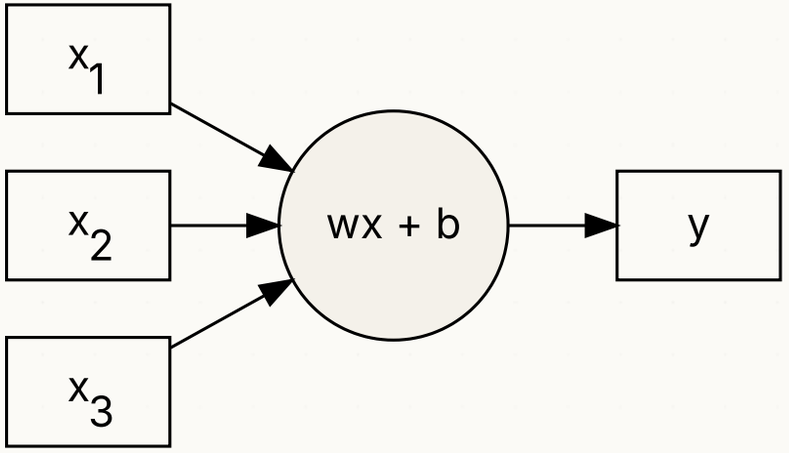
Let $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}$ — a row vector of three features.

Each feature deserves its own weight:

$$\mathbf{w} = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}$$

The combined signal is the **dot product**:

$$\mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2 + w_3 x_3$$



> **Note**
>
> Note that the dot product still returns a single number.

A worked social-science example

PREDICT TURNOUT IN THE 2024 UK GENERAL ELECTION

The 2024 UK General Election had a 59.7% turnout (lowest since 2001).

Imagine we want a model that predicts whether a voter actually turned out.

Three features, all normalised to $[0, 1]$:

- x_1: **age** (0 = 18, 1 = 80) — older voters turn out more.
- x_2: **income** (0 = £15k, 1 = £100k) — richer voters turn out more.
- x_3: **2019 turnout** (0 = no, 1 = yes) — past behaviour predicts.

A perceptron, with my guess at the weights:

- $w_1 = 1.5$
- $w_2 = 0.7$
- $w_3 = 2.0$
- $b = -1.5$ (baseline; reflects ~60% national turnout)

A *45*-year-old voter on *£40k* who *voted in 2019*:

- $x_1 = 27/62 \approx 0.44$
- $x_2 = 25/85 \approx 0.29$
- $x_3 = 1$

Forward pass:

$$\begin{aligned} y &= 1.5(0.44) + 0.7(0.29) + 2.0(1) + (-1.5) \\ &= 0.66 + 0.20 + 2.0 - 1.5 \\ &= 1.36 \end{aligned}$$

A "turnout score" of 1.36 so **likely to vote**.

Tip

Real turnout models built on **British Election Study** data look much like this — same features and fitted weights.

Wait a second...

STOP & SPOT IT

The prediction we just wrote out is:

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Does this look familiar?

Rename the weights:

$$y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \alpha$$

THIS IS **LINEAR REGRESSION**

Our perceptron *is* a linear regression model — we just relabelled the symbols.

Hopefully you have seen this before!

- The good news: you already know how the simplest neural network works.
- The bad news: a single linear neuron can only learn straight-lines...

To do anything more interesting we first need to **stack** neurons.

So why is everyone excited?

If a single neuron is just regression, what's all the fuss about?

A single neuron is linear

It can draw a *straight line* (or hyperplane) to separate two classes.

It cannot fit curves, stepwise changes, multiple gradients.

Most real social and computational problems are not linearly separable.

How do we make the model more capable?

- Stack many neurons in parallel — a **wide** layer.
- Stack many layers in series — a **deep** network.

That should give us more expressive models by having more parameters that can "interact"

But (!) stacking alone **cannot** add non-linearity

- We'll return to this

Some vocabulary, briefly

Node (or neuron)
: One computation: $\mathbf{w} \cdot \mathbf{x} + b$ (for now — we'll add one more piece soon).

Layer
: A group of nodes at the same "depth".

Input layer
: Your features $\mathbf{x}$. Not really a layer of computation — just the data going in.

Output layer
: The final layer; produces the prediction.

Hidden layer
: Any layer between input and output.

Width
: Number of nodes in a layer.

Depth
: Number of hidden layers.

Deep network
: Conventionally, ≥ 2 hidden layers. (1 hidden layer is "shallow".)

Stop and take stock

STOP & CHECK

TRY THIS — 5 MINUTES IN PAIRS

Your colleague says: *"A simple perceptron model mimics a neuron – it receives a signal, decides whether to amplify it, and passes it onto the next neuron."*

Is that right? What's missing?

A DEFENSIBLE ANSWER

Some of it is right – the neuron metaphor is genuinely useful when we think of "amplification" as weighting and biasing. But, the weight could make it smaller. And a simple perceptron *model* has only a single node, so there's no "passing it on" in this case.

Plenary · forward pass on paper

Plenary · forward pass on paper

≈ 20 MIN · PEN AND PAPER, IN PAIRS

You will compute three forward passes by hand.

THE MODEL

A linear perceptron with three inputs.

Weights: $w_1 = 0.4,\ w_2 = -0.3,\ w_3 = 1.1$ Bias: $b = 0.2$

For each of the following inputs, compute $y = \mathbf{w} \cdot \mathbf{x} + b$:

Input	x_1	x_2	x_3
A	1	2	0
B	0	0	1
C	0.5	1	-1

BONUS

We have written the model as an equation. Can you convert this into a functional form over $\mathbf{x}, \mathbf{w}, b$?

Plenary · answers

LINEAR PERCEPTRON OUTPUTS

- A: $0.4(1) - 0.3(2) + 1.1(0) + 0.2 = 0.4 - 0.6 + 0 + 0.2 = 0.0$
- B: $0.4(0) - 0.3(0) + 1.1(1) + 0.2 = 0 - 0 + 1.1 + 0.2 = 1.3$
- C: $0.4(0.5) - 0.3(1) + 1.1(-1) + 0.2 = 0.2 - 0.3 - 1.1 + 0.2 = -1.0$

BONUS $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$

We're going to be doing a lot of dot products this August!

Every layer of every neural network is built out of this exact operation — a dot product plus a bias. No matter how complicated the architecture, it will rely on this very basic maths.

Part 2 · Stacking neurons

Why one linear neuron isn’t enough — concretely

A single linear neuron gives:

$$y = w_1 x_1 + w_2 x_2 + b$$

As we’ve said, this is a **line** (or a hyperplane in higher dimensions).

PREDICT

Could a single neuron classify this dataset?

Two features x_1, x_2, two classes:

- Class A: points around $(0, 0)$ and $(1, 1)$
- Class B: points around $(0, 1)$ and $(1, 0)$

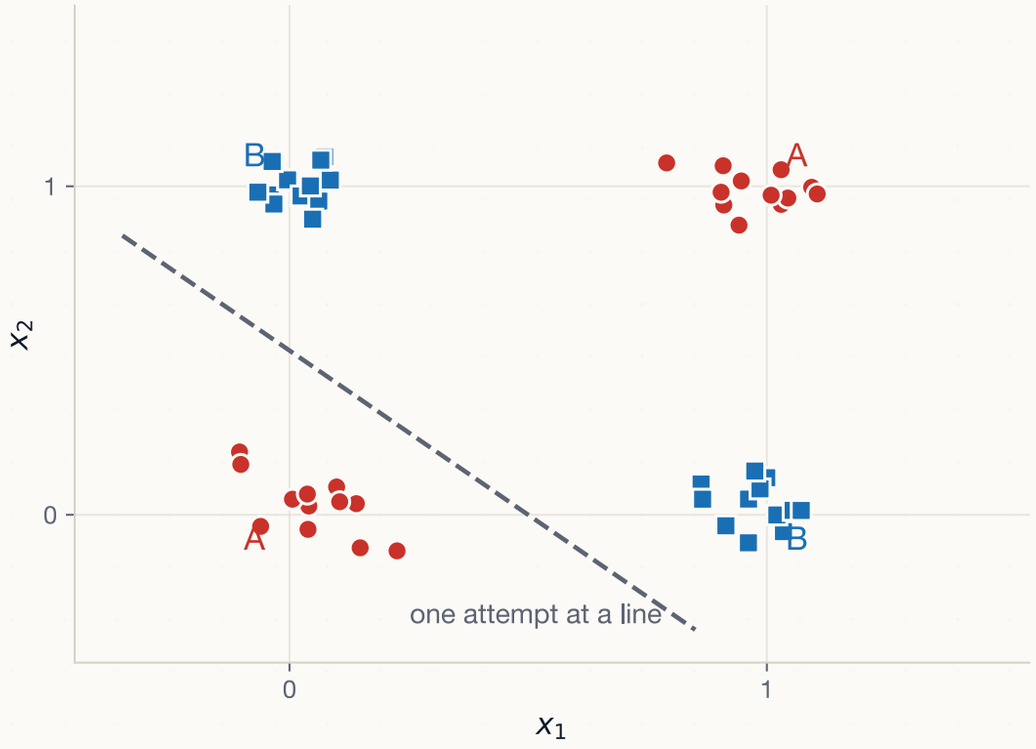
ANSWER

No. No single line separates A from B in this arrangement.

A linear perceptron *cannot* solve XOR. This is a result by Minsky & Papert that nearly ended neural networks in 1969

Seeing the problem

VISUALIZING THE XOR DATASET



Class A sits on two *opposite* corners, and Class B on the other two.

Wherever you draw a line, at least one cluster is placed on the wrong side. Datasets like this are called **not linearly separable**.

So let’s try stacking

THE NATURAL NEXT MOVES

We have one linear neuron. It can only draw lines.

What if we **stack** more of them?

Draw many parallel lines, then chain those into another layer.

Add more parallel lines that take the original lines as *inputs*!

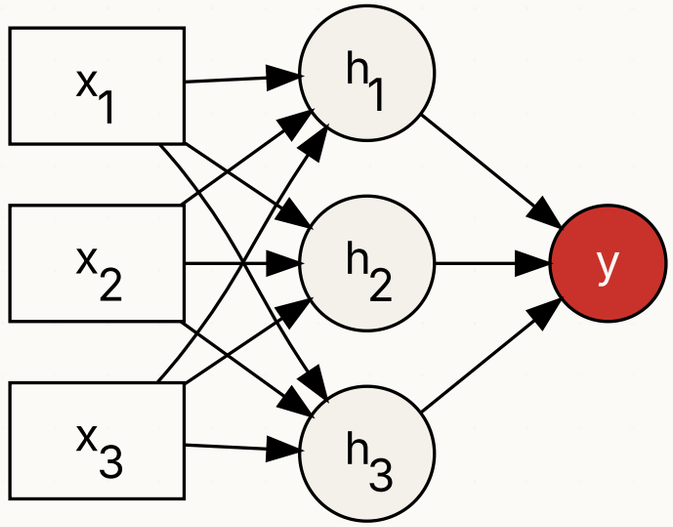
Not only can we learn *multiple representations* by using multiple neurons...

- Adding *layers* of neurons means our models can *interact* these representations

> **Note**
>
> Spoiler: unfortunately, this alone won’t be enough!

Width — multiple neurons in parallel

Take a layer of, say, three neurons that all see the same inputs:



- Each h_i has its **own** weights $\mathbf{w}^{(i)}$ and bias $b^{(i)}$.
- All h_i see the same inputs $\mathbf{x}$.
- The output node combines the h_i's, with yet another set of weights.

This is one **hidden layer of width 3**. Three nodes can — in principle — represent three different "concepts" extracted from $\mathbf{x}$. With non-linearities, they can each capture a *different* non-linear feature.

It’s easier (I promise!) to track weights with a matrix

For one hidden layer of three nodes, with three inputs each, we have **9 weights**. We can stack them into a matrix:

$$\mathbf{W} = \begin{bmatrix} w_1^{(1)} & w_2^{(1)} & w_3^{(1)} \\ w_1^{(2)} & w_2^{(2)} & w_3^{(2)} \\ w_1^{(3)} & w_2^{(3)} & w_3^{(3)} \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix}$$

Now the whole hidden layer’s pre-activation is **one matrix product**:

$$\mathbf{h}_{\text{pre}} = \mathbf{x}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

Shape check: $(1 \times 3) \cdot (3 \times 3) + (1 \times 3) \to (1 \times 3)$. ✅

> **Note**
>
> Don’t worry if the transpose looks fiddly — we’ll spend (a lot) more time on the linear algebra tomorrow.

Convention I'll use throughout the course

- **Samples are row vectors**: $\mathbf{x} \in \mathbb{R}^{1\times k}$.
- **Datasets are row-stacked**: $\mathbf{X} \in \mathbb{R}^{n\times k}$ — one observation per row.
- **Weight matrices**: $\mathbf{W} \in \mathbb{R}^{h\times k}$ for a layer of width h with k inputs.

So a layer's forward pass is $\mathbf{X}\mathbf{W}^\mathsf{T} + \mathbf{b}$, where biases *broadcast* across rows.

Tip

PyTorch follows this convention. Numpy can be made to. Some textbooks (e.g. Goodfellow et al.) use the opposite convention with column vectors. Don't be confused if you see $\mathbf{W}\mathbf{x}$ in one paper and $\mathbf{x}\mathbf{W}^\mathsf{T}$ in another — they're the same thing, rotated.

Depth — chaining layers

Take the hidden layer's output and feed it straight into a second layer (no transformation in between, for now):

$$\mathbf{h}^{(1)} = \mathbf{x}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)}$$

$$\hat{y} = \mathbf{h}^{(1)}\mathbf{w}^{(2)\mathsf{T}} + b^{(2)}$$

The whole network is the **composition** of these layers:

$$\hat{y} = \left(\mathbf{x}\,\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)}\right)\mathbf{w}^{(2)\mathsf{T}} + b^{(2)}$$

A "deep linear" network is just this pattern repeated — matmul → bias → matmul → bias → ...

- (*matmul* = "matrix multiplication")

Worked example — by hand

SETUP FOR A TWO-LAYER LINEAR NETWORK

$$\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}, \quad \mathbf{W}^{(1)} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad b^{(2)} = 1$$

STEP 1 — HIDDEN LAYER

$$\mathbf{h}^{(1)} = \mathbf{x}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)} = \begin{bmatrix} \underbrace{1\cdot 1 + 2\cdot 2 + 3\cdot 3}_{=14} & \underbrace{1\cdot 4 + 2\cdot 5 + 3\cdot 6}_{=32} \end{bmatrix} + \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 15 & 34 \end{bmatrix}$$

Worked example — by hand (2)

STEP 2 — OUTPUT

$$\hat{y} = \begin{bmatrix} 15 & 34 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 1 = 15 + 68 + 1 = \mathbf{84}.$$

You just ran a forward pass through a two-layer network by hand.

The operations never get more complicated than this. The matrices just get bigger!

Note

A real-world two-layer network for our voter-turnout problem might have 3 inputs, 64 hidden units, and 1 output — that's $64 + 1 = 65$ numbers in $\mathbf{b}$ (one bias per hidden unit, plus one for the output) and $3 \times 64 + 64 \times 1 = 256$ numbers in $\mathbf{W}$. Still the same operations, repeated.

Have we fixed the linearity constraint?

LET'S LOOK AT THE NETWORK END-TO-END

The full forward pass:

$$\hat{y} = \left(\mathbf{x}\,\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)}\right)\mathbf{w}^{(2)\mathsf{T}} + b^{(2)}$$

Expand the brackets:

$$\hat{y} = \mathbf{x}\,\underbrace{\mathbf{W}^{(1)\mathsf{T}}\mathbf{w}^{(2)\mathsf{T}}}_{\text{another matrix}} + \underbrace{\mathbf{b}^{(1)}\mathbf{w}^{(2)\mathsf{T}} + b^{(2)}}_{\text{another scalar}}$$

THE COLLAPSE

This is **still just** $\hat{y} = \mathbf{x}\mathbf{W}^{*} + b^{*}$ for *some* combined matrix and bias.

Two linear layers collapse into one linear layer. **A 100-layer linear network is mathematically identical to a 1-layer linear network.** Depth, on its own, buys us nothing.

We achieve non-linearity by using activation functions

THE FIX: A SMALL NON-LINEAR STEP BETWEEN LAYERS

Add a non-linear function σ between each linear layer:

$$\mathbf{h}^{(1)} = \sigma\left(\mathbf{x}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)}\right), \qquad \hat{y} = \mathbf{h}^{(1)}\mathbf{w}^{(2)\mathsf{T}} + b^{(2)}$$

Because σ is non-linear, we can no longer combine the two matrix multiplies into one. The network can now **bend** between the layers

- Enabling us to learn curves, kinks, decision boundaries that a line cannot capture

Theoretically, this is the move that really matters

A first taste of activation functions

COMMON ACTIVATION FUNCTIONS

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Squashes z to $(0, 1)$.
- Was the standard activation for decades.
- Still used in the output of binary classifiers.

tanh

$$\sigma(z) = \tanh(z)$$

- Squashes z to $(-1, 1)$.
- A zero-centred version of sigmoid.
- Used in RNNs (Day 9).

ReLU

$$\sigma(z) = \max(0, z)$$

- Returns z if positive, 0 otherwise.
- Simple. Fast. Wildly effective.
- The default in modern networks since ~2012.

Tip

We'll return to these activation functions in proper detail on **Day 5** (PyTorch implementation), and again on Days 6, 9, 10.

For now just know that *adding any of these between linear layers* is what makes flexible deep learning possible.

Activity · predict the output

Activity · predict the output

≈ 20 MIN · PEN AND PAPER, IN PAIRS

A *TWO-LAYER* NETWORK. IDENTITY ACTIVATION THROUGHOUT.

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1.0 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad b^{(2)} = 0.5$$

Compute $\hat{y}$ for $\mathbf{x} = \begin{bmatrix} 2 & 4 \end{bmatrix}$.

BONUS

What changes if we *swap* the bias term $b^{(2)}$ from 0.5 to -0.5? Does it change $\hat{y}$? By how much?

Activity · answer

HIDDEN LAYER PRE-ACTIVATION

$$\mathbf{x}\mathbf{W}^{(1)\mathsf{T}} = \begin{bmatrix} 2 & 4 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 1-2 & 2+2 \end{bmatrix} = \begin{bmatrix} -1 & 4 \end{bmatrix}$$

Add bias: $\begin{bmatrix} -1 & 4 \end{bmatrix} + \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 5 \end{bmatrix}$

OUTPUT

$$\hat{y} = \begin{bmatrix} -1 & 5 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + 0.5 = (-2) + (-5) + 0.5 = -6.5$$

BONUS Changing $b^{(2)}$ from 0.5 to -0.5 subtracts 1 from $\hat{y}$ — answer becomes -7.5. A linear shift; nothing else changes.

Part 3 · Where do weights come from?

So far, we've assumed all the weight and bias values

In every example so far, I just wrote down values for $\mathbf{W}$ and $\mathbf{b}$.

QUESTION

For real data, how do we choose those numbers?

ANSWER

We don't! The model uses an algorithm to update and *find* a good solution.

The numbers should be those that make $\hat{y}$ match y_{true} as closely as possible across our training data.

- This is why deep learning is a form of machine learning more generally

What we need

Two pieces of standard ML and a DL specific algorithm:

1. A way to measure "how wrong"

A function — the **loss** — that compares prediction to truth.

For continuous outcomes: squared error. For probabilities: cross-entropy. We'll meet both.

2. A way to know which way to nudge each parameter

For each weight, "if I increase this by ϵ, does the loss go up or down? By how much?"

That's a **gradient**.

3. A way to compute those gradients automatically

For a network with millions of parameters, we can't differentiate by hand.

We need an algorithm that walks the network's structure and computes gradients itself.

This is **backpropagation**.

The next two days, mapped

Tomorrow (Day 3)

We cover the **tools** we need:

- Linear algebra (vectors, matmul)
- Calculus (derivatives and the chain rule)
- **Computational graphs** — a way to draw any computation that scales to billions of operations.

Thursday (Day 4)

We **use** those tools.

- Define a loss
- Compute the gradient of the loss w.r.t. every weight, *automatically*
- Update the weights.

Repeat.

By the end of Thursday's lab, you'll have written **backpropagation** by hand!

One more distinction

TWO FLAVOURS OF MODELLING

Predictive (discriminative)

- Estimate $P(y \mid \mathbf{x})$.
- "Given features, predict the label."
- Voter turnout, sentiment classification, image labelling, civil-war forecasting.
- Most of our first six lectures sit here.

Generative

- Estimate $P(\mathbf{x} \mid y)$ — and sample from it.
- "Make a new example."
- DALL·E, ChatGPT, synthetic survey data, MIDAS-style imputation.
- From Day 7 onwards we'll focus here.

The same machinery — matmul + activation + loss + gradient — drives both. The differences are in:

- The shape of the output.
- The loss function.
- The training data (labelled or unlabelled).

Take stock

WHERE YOU ARE AT THE END OF DAY 2

You can now:

1. Define a neural network in plain language and as a diagram.
2. Compute a forward pass through a small network **by hand**.
3. Name the parts: nodes, layers, weights, biases, activations, depth, width.
4. Explain to a non-specialist why one perceptron is “just” linear regression.
5. State precisely what you *don’t* yet know: where the weights come from.

Tip

That is the conceptual core of the course. The next ten lectures expand outwards from this — but the centre is here.

Lab brief

Today’s lab — 90 minutes

GOALS

In a Colab notebook, you will:

1. Implement a perceptron *as a Python function*, with explicit weights and bias.
2. Run a forward pass on a real dataset (a small survey-data sample we’ll provide).
3. Implement a **two-layer network** on the same data.
4. Compare predictions to a simple logistic regression baseline. Look at where the network wins and where it doesn’t.

Deliverable: a one-page notebook with predictions for a hold-out set, plus a short paragraph: “Where does the two-layer model beat the perceptron? Where doesn’t it?”

Tip

This lab does *not* involve PyTorch yet. We’re keeping the operations in plain numpy so the algebra you just did stays visible.

See you tomorrow

LECTURE 3 — *COMPUTATIONAL GRAPHS & THE MATHS OF AI*

Optional pre-reading tonight: 3Blue1Brown, *Essence of Linear Algebra* chapters 1–3 (≈30 min). The intuitions there will save you a lot of grief tomorrow.